

# Node

## Packet-decomp Module

This module takes a 32-bit packet from the FIFO and decomposes it into 4 8-bit packets that are then sent to the routers. It uses the following FSM to determine when to read from the FIFO and when to assert payload-avail.

Inputs  
clock  
reset\_n  
[31:0] packet  
packet\_avail  
requested

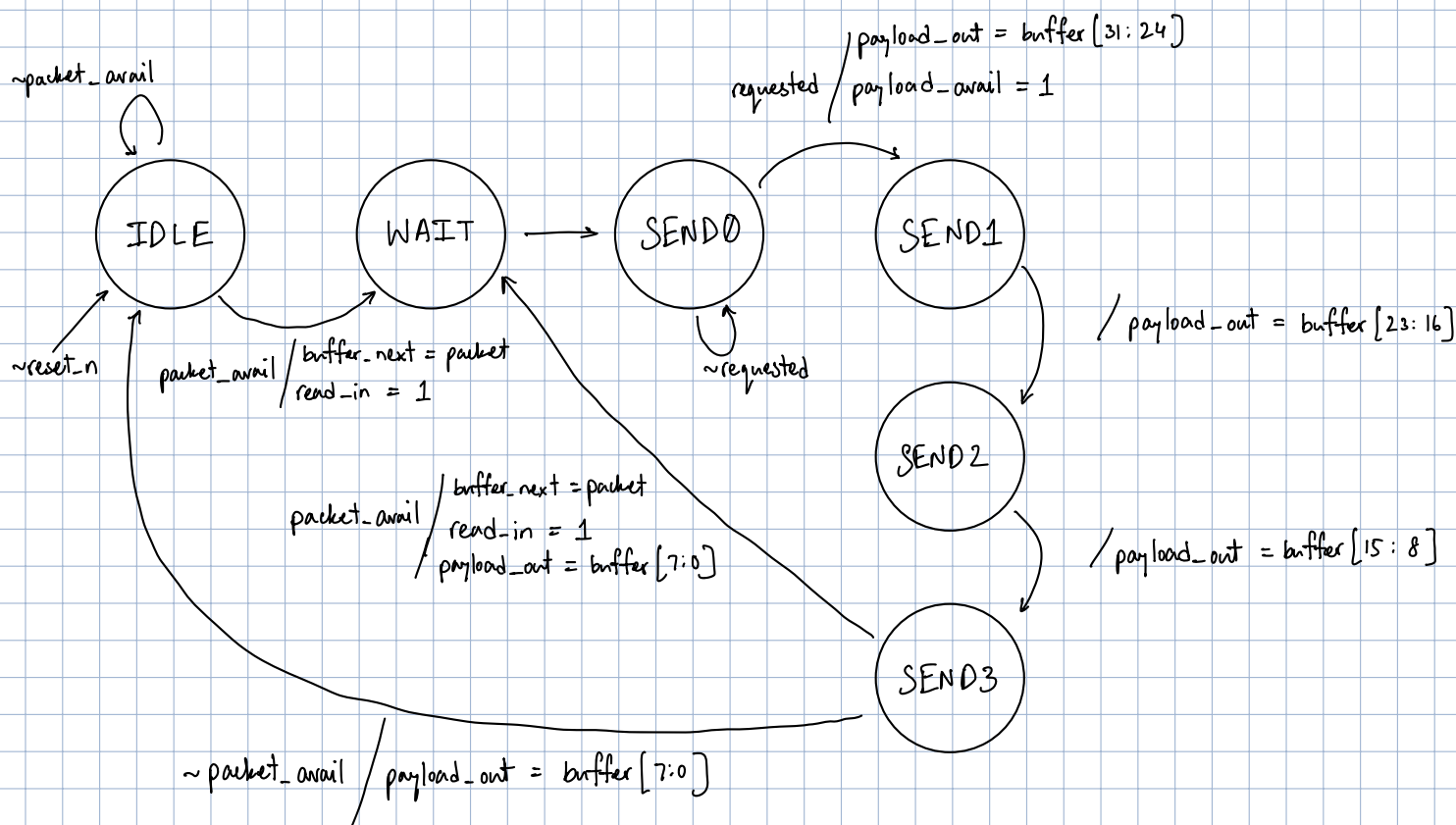
Outputs  
[7:0] payload\_avail  
payload\_out  
read\_in

Internal Logic Variables  
[31:0] buffer → stores the 32-bit packet sent to the Node  
[31:0] buffer\_next → captures the next value of buffer based on the state. This value is sent to buffer at each passage of the clock

FSM:

Assume the following default values if an output isn't specified for a given transition.

Default Values {  
① buffer\_next = buffer  
② payload\_avail = 1'b0  
③ payload\_out = 8'd0  
④ read\_in = 1'b0



Note that the state value and buffer value are stored in registers. These are the only components in my datapath, and are updated with next\_state and buffer\_next at each passage of the clock. Both next\_state and buffer\_next are determined combinationaly based on the current state and inputs to the FSM.

## Packet\_reconstruct Module

This module takes 4 8-bit payloads sequentially and then assembles them back into a 32-bit packet. It uses the following FSM to track which byte is next and to assert packet\_avail when ready.

Inputs  
clock  
reset\_n  
[7:0] payload

Outputs  
[31:0] packet  
packet\_avail  
free\_inbound

### Internal Logic Variables

buffer → stores the reconstructed 32-bit packet along with the intermediate steps during reconstruction

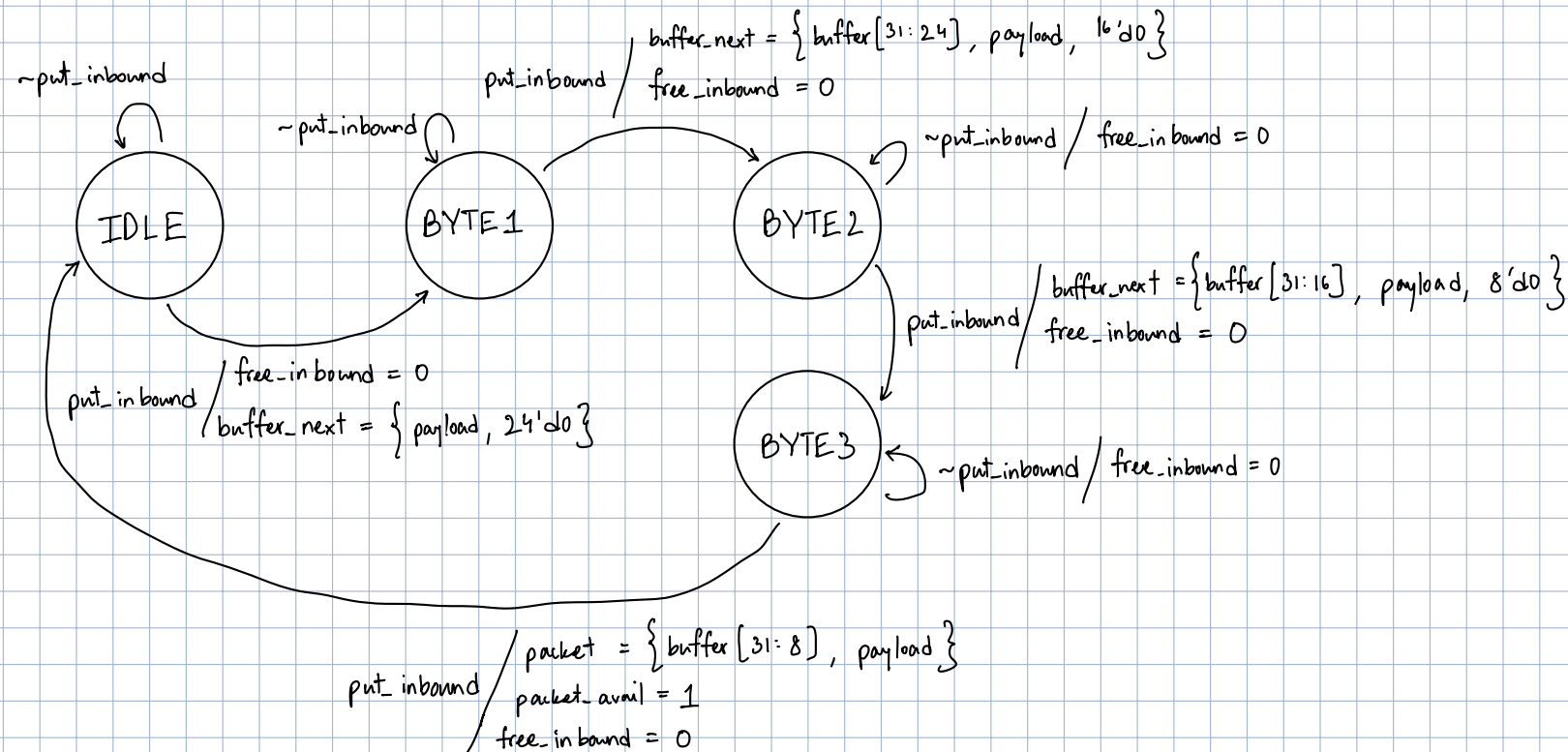
buffer\_next → captures the next value of buffer that is produced as a result of the state. This value is updated into buffer at the posedge of the clock.

### FSM:

Assume the following default values if an output isn't specified on a transition:

Default Values {

- ① buffer\_next = buffer
- ② packet = 32'd0
- ③ packet\_avail = 1'b0
- ④ free\_inbound = 1'b1



Note that the curr\_state and buffer values are stored in registers. These are the only components that are in the datapath for this module. They are updated with next\_state and buffer\_next (both of which are determined combinatorially) at each posedge of the clock.

## FIFO Module

The FIFO is a 4-deep queue that stores 32-bit words and supports one read and/or one write per cycle. It outputs the oldest output combinationaly when not empty and adds new values on the posedge of the clock when writing. It uses two pointers to keep track of the front and back of the FIFO.

### Inputs

clock  
reset\_n  
[WIDTH-1:0] data\_in ← (always 32 bits for this project)  
we  
re

### Outputs

full  
empty

[WIDTH-1:0] data\_out ← (always 32 bits for this project) → [WIDTH-1:0] data\_out

### Internal Logic Variables

[3:0][WIDTH-1:0] buffer → this is where the queue elements are actually stored

[1:0] read\_ptr → location of front of queue

[1:0] write\_ptr → location where next element can be added to queue

[2:0] counter → keeps track of size of FIFO

[1:0] counter\_state → keeps track of what operation (read, write, etc.) is actually happening in the queue and is used to modify the counter based on the operation

→ counter\_state = { we & ~full, re & ~empty }

→ 01 : read only, 10 : write only, 11 : both read and write

### FSM:

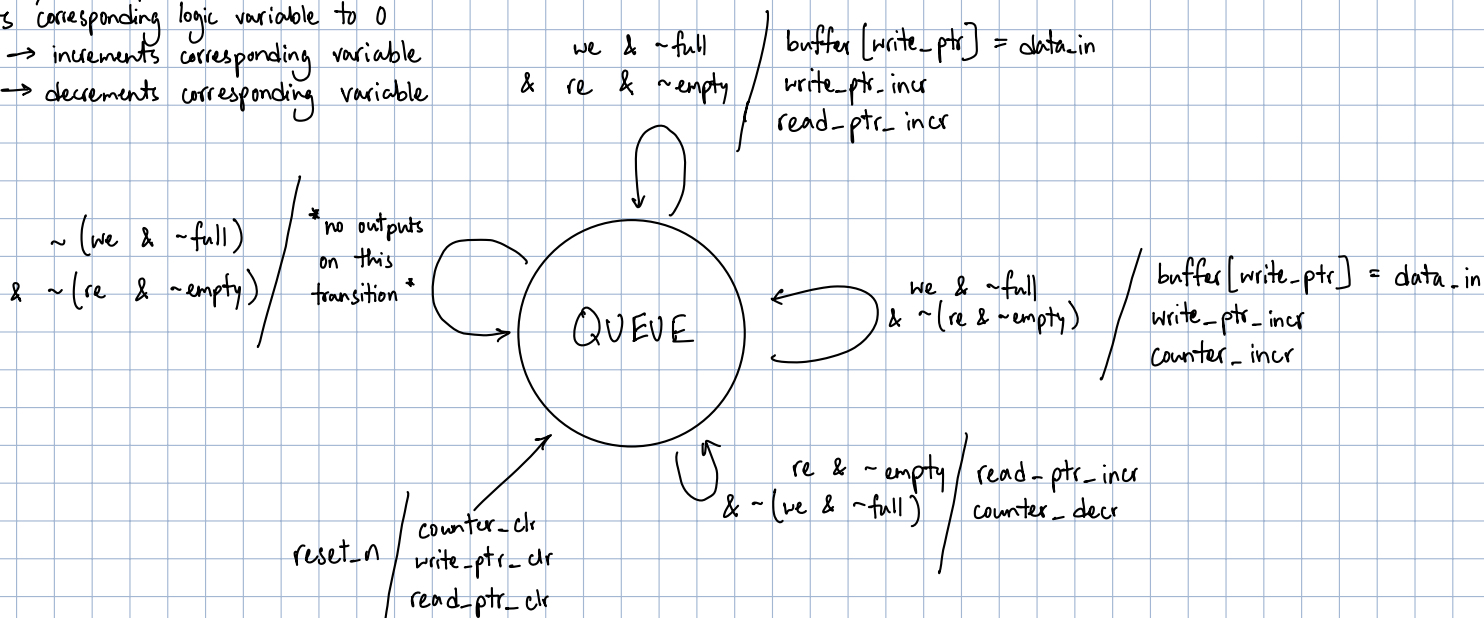
#### Signal Definitions:

write\_ptr\_clr, read\_ptr\_clr, counter\_clr

→ sets corresponding logic variable to 0

"\_incr → increments corresponding variable

"\_decr → decrements corresponding variable



data\_out, full, and empty are determined combinationaly:

assign data\_out = buffer[read\_ptr]  
assign full = (counter == 3'd4)  
assign empty = (counter == 3'd0)

# Node Datapath Schematic

